RV32IM Core & SoC Subsystem

Architecture and Integration

RISCFORGE 1TOPS | ARCH-INTEG-v5.0

Introduction

This document describes the RISCFORGE 1TOPS SoC subsystem and how the RV32IM processor core is connected to it. The earlier `riscv_core_stub.sv` placeholder has been removed; the subsystem now instantiates the real 5-stage pipelined core directly, with no adapter wrapper between them.

It covers the SoC structure, the memory-mapped I/O map, the reset scheme, the core's internal modules, the CSR file, and the hybrid memory interface that joins the core's single data port to two very different destinations.

System Architecture

The subsystem is a single-core microcontroller: one processor, two on-chip SRAMs, a boot ROM, an AXI4-Lite crossbar, an AXI-to-APB bridge, six APB peripherals, and a SPI-flash boot controller.

Block Diagram

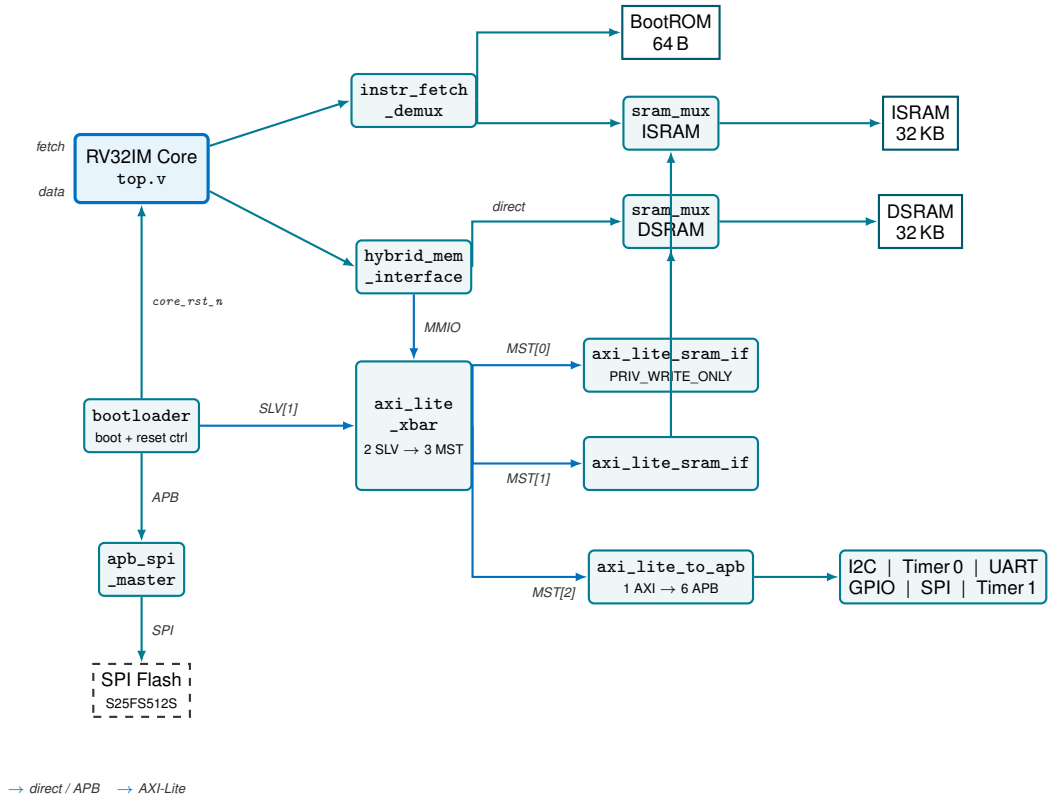


Figure 1: SoC subsystem block diagram. Both SRAM paths from the core are direct; only MMIO traffic crosses the AXI-Lite crossbar.

Structure

The design separates traffic by how fast it must be, not by what it is.

- **The core's two memory ports never touch the crossbar.** Instruction fetch reaches ISRAM through `instr_fetch_demux` and `sram_mux` port 0. Loads and stores reach DSRAM through `hybrid_mem_interface` and `sram_mux` port 0. Both are combinational paths into the SRAM, so neither pays crossbar latency.
- **Only peripheral traffic goes over AXI.** When the core addresses the MMIO window, `hybrid_mem_interface` converts that single access into an AXI-Lite transaction on crossbar slave port 0.
- **The bootloader is the second bus master.** It sits on slave port 1 and uses the crossbar only during boot, to fill both SRAMs. It also owns the reset controller.
- **Each SRAM has two ports.** `sram_mux` arbitrates between the core (port 0, priority) and the AXI bridge (port 1). The two are never active together: the core is held in reset while the bootloader writes, and the ISRAM AXI port is locked out once boot completes.

Blocks

Block	Count	Role
`top` (RV32IM core)	1	5-stage pipelined processor, Machine mode only
`hybrid_mem_interface`	1	Routes core data accesses to DSRAM or to MMIO
`boot_rom`	1	16-word reset-vector ROM, reachable only by the core
`imem_sram_wrapper`	1	32 KB instruction SRAM (ISRAM)
`sram_sp`	1	32 KB data SRAM (DSRAM)

Block	Count	Role
sram_mux	2	2:1 priority mux, one per SRAM
axi_lite_sram_if	2	AXI-Lite to SRAM bridge, one per SRAM
instr_fetch_demux	1	Sends each fetch to the BootROM or to ISRAM
axi_lite_xbar	1	2 slave ports to 3 master ports
axi_lite_to_apb	1	1 AXI-Lite port to 6 APB slaves
bootloader	1	Boot FSM plus reset controller
APB peripherals	6	I2C, Timer 0, UART, GPIO, SPI, Timer 1

Interconnect

Two masters share the crossbar: the core on slave port 0 and the bootloader on slave port 1. Three master ports lead to ISRAM, DSRAM and the APB bridge.

Listing 1: Crossbar configuration

```
NUM_SLV_PORTS  = 2      // [0] core (via hybrid_mem_interface), [1] bootloader
NUM_MST_PORTS  = 3      // [0] ISRAM, [1] DSRAM, [2] APB bridge
MAX_MST_TRANS  = 4
MAX_SLV_TRANS  = 4
LatencyMode    = axi_pkg::CUT_ALL_AX
```

What `CUT_ALL_AX` Does

The crossbar contains one demux per slave port and one mux per master port. `LatencyMode` chooses which channels get a pipeline register at those points:

```
CUT_ALL_AX = DemuxAw | DemuxAr | MuxAw | MuxAr    // axi_pkg.sv:475
```

Only the address channels (AW, AR) are cut, and they are cut twice — once at the demux and once at the mux. The write-data, write-response and read-data channels pass straight through. Address decoding is the longest combinational path in a crossbar, so cutting it helps timing at the cost of two cycles of address latency.

Note

Because AW is registered and W is not, the crossbar can accept an *address one cycle before the data*. Any master attached to it must handle the two channels completing independently.

Important

Only MMIO traffic crosses the crossbar. Both SRAM paths from the core are direct: instruction fetch reaches ISRAM through `instr_fetch_demux` and `sram_mux` port 0, and loads and stores reach DSRAM through the combinational SRAM side of `hybrid_mem_interface` and `sram_mux` port 0. This is deliberate — routing SRAM traffic through the crossbar would add several cycles to every access.

Memory-Mapped I/O

Connection Map

Source	Path	Destination
Core fetch port	instr_fetch_demux	BootROM, 0x0000_0000
Core fetch port	instr_fetch_demux → sram_mux port 0	ISRAM, 0x0001_0000

Source	Path	Destination
Core data port	`hybrid_mem_interface` → `sram_mux` port 0	DSRAM, `0x0002_0000`
Core data port	`hybrid_mem_interface` → AXI-Lite	Crossbar SLV[0], MMIO window
Bootloader	AXI-Lite master, `awprot[0] = 1`	Crossbar SLV[1], SRAM writes only
Bootloader	APB slave port	From APB bridge slot 4
Bootloader	APB master port	`apb_spi_master`
Crossbar MST[0]	`axi_lite_sram_if` (`PRIV_WRITE_ONLY`) → `sram_mux` port 1	ISRAM
Crossbar MST[1]	`axi_lite_sram_if` → `sram_mux` port 1	DSRAM
Crossbar MST[2]	`axi_lite_to_apb`	Six APB peripherals

The bootloader additionally drives the three internal reset outputs `core_rst_ni`, `periph_rst_ni` and `spi_rst_ni`. Its APB master port is FSM-driven during boot and becomes a transparent passthrough at runtime.

Address Map

Crossbar Regions

Region	Range	Size	Notes
ISRAM	`0x0001_0000` – `0x0001_8000`	32 KB	Privilege-gated writes
DSRAM	`0x0002_0000` – `0x0002_8000`	32 KB	
APB	`0x4000_0000` – `0x5000_0000`	256 MB	Peripheral space

Core-Only Region

Region	Range	Size	Notes
BootROM	`0x0000_0000` – `0x0000_0040`	64 B	16 words, read-only, not on the crossbar

APB Peripherals

Slot	Peripheral	Range	Size
APB[0]	I2C	`0x4000_0000` – `0x4000_1000`	4 KB
APB[1]	Timer 0	`0x4000_1000` – `0x4000_2000`	4 KB
APB[2]	UART	`0x4000_2000` – `0x4000_3000`	4 KB
APB[3]	GPIO	`0x4000_3000` – `0x4000_4000`	4 KB
APB[4]	SPI	`0x4000_4000` – `0x4000_5000`	4 KB
APB[5]	Timer 1	`0x4000_5000` – `0x4000_6000`	4 KB

Note

The BootROM is wired straight to the fetch demux and is not on the crossbar, so no bus master can reach the reset vector. ISRAM writes over AXI are privilege-gated: only transactions with `awprot[0] = 1` are accepted, and unprivileged writes return SLVERR.

Memories

BootROM — `0x0000_0000 - 0x0000_003F`

64 bytes, 16 words, read-only. Reachable only through the instruction fetch demux; it is not on the AXI crossbar, so no bus master can touch the reset vector.

ISRAM — `0x0001_0000 - 0x0001_7FFF`

32 KB, 8192 words. The core fetches from it directly, read-only. AXI writes require `awprot[0] = 1`; unprivileged writes return SLVERR. After boot completes the AXI port is locked out entirely.

DSRAM — `0x0002_0000 - 0x0002_7FFF`

32 KB, 8192 words. Core loads and stores reach it directly; the bootloader writes it over AXI.

I2C Master — APB[0], `0x4000_0000`

Offset	Name	Width	Access	Description
`0x00`	`PRE`	16	R/W	Clock prescaler; SCL $=$ PCLK$/(5\times$(PRE$+1))$
`0x04`	`CTR`	8	R/W	`[7]` EN core enable, `[6]` IEN interrupt enable
`0x08`	`RXR`	8	R	Received byte
`0x0C`	`SR`	8	R	`[7]` RxACK (0 $=$ ACK), `[6]` Busy, `[5]` AL arbitration lost, `[1]` TIP transfer in progress, `[0]` IF interrupt flag
`0x10`	`TXR`	8	W	Transmit byte or slave address
`0x14`	`CR`	8	W	`[7]` STA start, `[6]` STO stop, `[5]` RD read, `[4]` WR write, `[3]` ACK value, `[0]` IACK interrupt acknowledge

Timer 0 — APB[1], `0x4000_1000`

Two independent timers (`TIMER_CNT = 2`). Timer N sits at offset $N\times$ `0x10`.

Offset	Name	Width	Access	Description
`0x00`	`TIMER_VAL`	32	R/W	Timer 0 current value
`0x04`	`TIMER_CTRL`	32	R/W	`[0]` EN enable, `[5:3]` PRESCALER (0 $=$ bypass)
`0x08`	`TIMER_CMP`	32	R/W	Compare value; resets the timer on match
`0x10`	`TIMER_VAL`	32	R/W	Timer 1 current value
`0x14`	`TIMER_CTRL`	32	R/W	Timer 1 control
`0x18`	`TIMER_CMP`	32	R/W	Timer 1 compare value

IRQ outputs: `[2N]` $=$ overflow, `[2N+1]` $=$ compare match. A compare interrupt is suppressed while `TIMER_CMP` is zero.

UART — APB[2], `0x4000_2000`

16550-compatible. `apb_uart` takes a 3-bit word address, `PADDR[4:2]`, giving eight registers.

Offset	Name	Width	Access	Description
0x00	RBR/THR/DLL	8	R/W	Receive buffer (R), transmit hold (W), or divisor latch LSB when DLAB = 1
0x04	IER/DLM	8	R/W	Interrupt enable, or divisor latch MSB when DLAB = 1. [0] ERBFI, [1] ETBEI, [2] ELSI, [3] EDSSI
0x08	IIR/FCR	8	R/W	Interrupt identification (R) / FIFO control (W)
0x0C	LCR	8	R/W	[7] DLAB, [5:3] parity, [2] stop bits, [1:0] word length
0x10	MCR	8	R/W	Modem control
0x14	LSR	8	R	[0] DR data ready, [5] THRE, [6] TEMT
0x18	MSR	8	R	Modem status
0x1C	SCR	8	R/W	Scratch register

GPIO — APB[3], 0x4000_3000

32 pins, lowRISC reggen-generated register file (BlockAw = 11).

Offset	Name	Access	Description
0x000	INFO	R	[9:0] GPIO count, [19:10] version
0x004	CFG	R/W	[0] global interrupt mode, [1] pin-level interrupt mode
0x008	GPIO_MODE_0	R/W	2-bit mode per GPIO[0..15]
0x00C	GPIO_MODE_1	R/W	2-bit mode per GPIO[16..31]
0x010	GPIO_EN	R/W	Output enable per pin
0x014	GPIO_IN	R	Sampled input value
0x018	GPIO_OUT	R/W	Output data value
0x01C	GPIO_SET	W	Write 1 to set output pins
0x020	GPIO_CLEAR	W	Write 1 to clear output pins
0x024	GPIO_TOGGLE	W	Toggle output pins
0x028	INTRPT_RISE_EN	R/W	Rising-edge interrupt enable
0x02C	INTRPT_FALL_EN	R/W	Falling-edge interrupt enable
0x030	INTRPT_LVL_HIGH_EN	R/W	Level-high interrupt enable
0x034	INTRPT_LVL_LOW_EN	R/W	Level-low interrupt enable
0x038	INTRPT_STATUS	R/W1C	Interrupt status
0x03C	INTRPT_RISE_STATUS	R/W1C	Rising-edge interrupt status
0x040	INTRPT_FALL_STATUS	R/W1C	Falling-edge interrupt status
0x044	INTRPT_LVL_HIGH_STS	R	Level-high interrupt status
0x048	INTRPT_LVL_LOW_STS	R	Level-low interrupt status

SPI Master — APB[4], 0x4000_4000

Reached through the bootloader's APB mux. During boot the boot FSM drives it; at runtime the APB bridge passes through.

Offset	Name	Width	Access	Description
0x00	STATUS	32	R	[0] TX full, [1] TX empty, [4] RX full, [5] RX empty, [8] active chip select
0x04	CLKDIV	8	R/W	$f_{SPI} = f_{CLK}/(2\times(\text{CLKDIV}+1))$
0x08	SPICMD	32	R/W	Command value, MSB-aligned
0x0C	SPIADR	32	R/W	Address value, MSB-aligned
0x10	SPILEN	32	R/W	[5:0] CMDLEN, [13:8] ADRLEN, [31:16] DATALEN, all in bits
0x14	SPIDUM	32	R/W	[15:0] dummy cycles for reads, [31:16] for writes

Offset	Name	Width	Access	Description
0x18	TXFIFO	32	W	Push TX FIFO
0x20	RXFIFO	32	R	Pop RX FIFO
0x24	INTCFG	32	R/W	Interrupt configuration
0x28	INTSTA	32	R	Interrupt status

Important

SPICMD and SPIADR are MSB-aligned. spi_master_tx always shifts out from bit 31 downward regardless of the configured length, so a 24-bit address must be placed in [31:8].

Timer 1 — APB[5], 0x4000_5000

Identical to Timer 0: two timers, same register layout, same offsets. This slot previously held the CLIC, which has been removed from the design.

Reset Scheme

Two external reset pins enter the subsystem. The bootloader derives three internal domains from them.

Signal	Source	Function
hard_rst_ni	External	Cold boot. SRAMs cleared, image re-fetched over SPI.
soft_rst_ni	External	Warm boot. SRAM contents kept, SPI fetch skipped.
sys_rst_n	Derived	hard_rst_ni & soft_rst_ni. Crossbar, APB bridge, SRAM bridges and muxes.
core_rst_n	Bootloader	Core, BootROM, hybrid_mem_interface, instr_fetch_demux. Low for the whole boot.
periph_rst_n	Bootloader	I2C, Timer 0, Timer 1, UART, GPIO.
spi_rst_n	Bootloader	apb_spi_master. Released early so the SPI IP works during boot.

Release Order

Signal	Released At	State of the System
spi_rst_n	cycle 10	SPI IP usable; core and peripherals still held
core_rst_n	cycle 121,633	Clean-slate pulse complete, image resident in ISRAM and DSRAM
periph_rst_n	cycle 121,633	Released on the same cycle as the core

Important

Which reset a block uses matters. instr_fetch_demux must be reset by core_rst_n, not sys_rst_n. The demux has one register, rom_accessed_q. Both memories return data one cycle after the address, so this register remembers which memory was addressed last cycle in order to route the returning data correctly. It has to reset to "BootROM", because the core's first fetch targets the reset vector.
The two resets release about 121,000 cycles apart. With sys_rst_n the register leaves reset correctly at cycle 10, but on cycle 11 it clocks in core_req_i & sel_rom, which is zero because the core is still in reset and not fetching. It stays zero for the whole boot, so when the core finally fetches, the mux selects ISRAM and the BootROM's first instruction is thrown away.
The general rule: if a register's reset value must still be valid when a different block starts running, it has to share that block's reset.

Boot Time

Phase	Cycles	Scales With
SRAM clear (16384 words)	≈ 81,900	Memory size
Flash power-up wait	30,000	Fixed, 300 µs
SPI image fetch	≈ 9,700	Image size, ≈646 cycles/word
Clean-slate reset pulse	6	Fixed
Total cold boot	**121,634**	

Why the Clear Costs So Much

The clear runs from cycle 10 to 81,930: 81,920 cycles for 16,384 words, which is exactly 5.0 cycles per word. Two things drive that.

It clears the whole memory, not the image. $8192 + 8192 = 16384$ words. The firmware image is 15 words. The cost scales with memory size, so it is roughly 99% of boot time however small the program is.

Each word costs five cycles because writes are serialised. The bootloader waits for the write response before issuing the next request:

Listing 2: Serialised clear loop

```
S_CLEAR_SRAM: begin
  if (axi_resp_i.b_valid)          // wait for THIS write before the next
    state_d = (clear_cnt_q + 1 < TOTAL_CLEAR_WORDS) ? S_CLEAR_SRAM : ... ;
end
```

The five cycles cover issuing AW and W, the crossbar's AW register, the bridge's write state where the SRAM is actually written, the response state returning `b_valid`, and the bootloader latching that response.

Note

Making it faster. Merging the clear into the image load saves nothing — the cost is the 16,384 writes themselves, not the fact that there are two passes. Three things do help: clear only DSRAM (ISRAM is fully overwritten by the image and a correct program never fetches uninitialised instruction memory), pipeline the loop to use the four outstanding transactions the crossbar already allows, or add a small zeroing counter inside the SRAM macro so the clear needs no bus traffic at all. Together these take the clear from about 82,000 cycles to roughly 8,000.

RISC-V Core Modules

The core is a 5-stage in-order pipeline — IF, ID, EX, MEM, WB — with full forwarding and an active-low reset convention throughout.

Module	Function
`top.v`	Core top level. Connects the five stages, the hazard unit, the CSR file and the trap controller, and contains the 16-wide interrupt priority encoder.
`fetch_cycle.v`	IF stage and the IF/ID register. Holds the next-PC mux and instantiates `PC`, `PC_Adder` and `imem_controller`. Detects misaligned-PC exceptions.
`decode_cycle.v`	ID stage and the ID/EX register. Instantiates `Register_File`, `Control_Unit_Top` and `Sign_Extend`.
`execute_cycle.v`	EX stage and the EX/MEM register. Instantiates `ALU`, `JB_unit` and `rv32m_unit`, plus the forwarding muxes and the CSR read-modify logic.

Module	Function
memory_cycle.v	MEM stage and the MEM/WB register. Instantiates ls_unit, checks load and store alignment, and drives the external lsu_mem_* bus.
writeback_cycle.v	WB result mux: ALU result, load data, PC+4 or CSR value.
imem_controller.v	Sits inside fetch_cycle. Drives the raw instruction-SRAM signals, pre-fetches PC_next so the memory latency is hidden, and gates a NOP into the pipeline during the one-cycle drain after a branch or trap redirect.
ls_unit.v	Sits inside memory_cycle. Formats load results (sign or zero extension, byte select) and builds store data alignment and byte strobes.
PC.v	Program counter register. Carries the RESET_PC parameter.
PC_Adder.v	PC + 4 adder.
ALU.v	Integer ALU.
ALU_Decoder.v	Maps funct3 and funct7 to ALU control.
Main_Decoder.v	Maps opcode to the top-level control bundle.
Control_Unit_Top.v	Wraps Main_Decoder and ALU_Decoder.
Sign_Extend.v	Immediate generation for every RV32I format.
Register_File.v	32 by 32-bit architectural registers with write-through forwarding. Commits on WE3 && A3 != 0.
JB_unit.v	Branch and jump condition evaluation.
Mux2.v, Mux_3_by_1.v	Generic datapath multiplexers.
opcode_check.v	Legal-opcode detection; drives the illegal-instruction exception.
hazard_unit.v	Forwarding selects and stall/flush generation for all five stages.
csr_regfile.v	Machine-mode CSR file, performance counters and interrupt qualification.
wb_trap_mret_commit_control.v	Commits traps and MRET at writeback; produces the redirect target and the pipeline flush vector.
rv32m_unit.v	MUL and DIV dispatch, and the multi-cycle stall stallMUL.
rv32m_mul.v	Booth-encoded Wallace-tree multiplier.
rv32m_div.v	Sequential divider.
booth_enc.v, pp_gen.v, csa.v	Booth encoder, partial-product generator and carry-save adder used by the multiplier.
hybrid_mem_interface.sv	Data-bus router. Not part of the pipeline — see Section 6.

Important

Elaboration reports width mismatches inside rv32m_mul.v: the .shift((2*i)) generate connection binds a 32-bit expression to a 6-bit port, and the csa instances bind 65-bit signals to 66-bit ports. These are pre-existing and were not exercised by the current test program, which contains no MUL or DIV. They should be resolved before the M-extension is relied on.

Hybrid Memory Interface

Why It Exists

The core has exactly one data-memory port. The SoC has two completely different places that port must be able to reach:

- **DSRAM** — physically adjacent, one-cycle latency, no protocol.
- **Peripherals** — behind an AXI-Lite crossbar and an APB bridge, several cycles away, with full handshaking.

Giving the core two separate ports would mean changing the core every time the SoC's memory map changed. Instead hybrid_mem_interface sits outside the core, presents one uniform port to it, and decides

per access which destination applies. The core never knows the difference; it issues a request and waits for `mem_stall` to drop.

Note

This module is *not* instantiated by `top.v`. It is instantiated by `riscforge_soc_subsystem.sv` and wired between the core's `lsu_mem_*` bus and the SoC.

Parameters

Parameter	Value in SoC	Meaning
`MEM_DEPTH`	8192	DSRAM size in words
`DMEM_BASE_BYTE`	`0x0002_0000`	DSRAM base; must match `xbar_addr_map[1]`
`MMIO_BASE`	`0x4000_0000`	Peripheral window base
`MMIO_LIMIT`	`0x4FFF_FFFF`	Peripheral window top; must match `xbar_addr_map[2]`
`axi_req_t`	SoC struct	AXI-Lite request type, passed in from the SoC
`axi_resp_t`	SoC struct	AXI-Lite response type

The AXI ports are type-parameterised rather than importing struct definitions from a package. This is the same pattern `axi_lite_sram_if`, `axi_lite_to_apb` and `bootloader` already use, and it lets the SoC's locally declared structs bind directly without a shared package dependency.

Internal Structure

The module is five pieces:

Block	Function
Address decoder	Compares `mem_addr` against the DSRAM and MMIO windows and produces `sram_req_active`, `axi_req_active` and `addr_fault`.
SRAM path	Purely combinational. Passes enable, write enable, byte mask, address and write data straight through to the DSRAM mux.
AXI write FSM	Four states. Drives AW and W, waits for both to be accepted, then waits for the B response.
AXI read FSM	Two states. Drives AR, then waits for R.
Aggregation layer	Combines everything into the three signals the core sees: `mem_stall`, `mem_err` and `mem_rdata`.

Address Decoding

Every access falls into exactly one of three cases:

Condition	Range	Action
`addr_in_sram_range`	`0x0002_0000` – `0x0002_7FFF`	Direct SRAM access
`addr_in_mmio_range`	`0x4000_0000` – `0x4FFF_FFFF`	AXI-Lite transaction to crossbar SLV[0]
`addr_fault`	anything else	`mem_err` raised, core takes an access fault

Important

ISRAM is deliberately not decoded here. A load or store to `0x0001_xxxx` raises an access fault. Instruction memory is written once at boot and is read-only to the program. The practical consequence is that firmware must keep every constant it loads in the data section the bootloader places in DSRAM — a constant left next to the code in ISRAM will trap.

Write FSM

State	Drives	Leaves When
`AXI_WRITE_IDLE`	`awvalid` and `wvalid` when a request is present	Depending on which readys came back: both, address only, or data only
`AXI_WAIT_ADDR`	`awvalid` (address still outstanding)	`aw_ready`
`AXI_WAIT_DATA`	`wvalid` (data still outstanding)	`w_ready`
`AXI_WAIT_RESP`	`bready`	`b_valid`

The two wait states exist because `CUT_ALL_AX` registers AW but not W, so the crossbar can accept them in different cycles. Each wait state must poll *its own* channel's ready signal; once that channel is accepted, both are done and the FSM moves to the response state.

Important

If the two ready checks are swapped — `AXI_WAIT_ADDR` polling `w_ready` and `AXI_WAIT_DATA` polling `aw_ready` — the core hangs on its very first MMIO access. The crossbar takes the address first and then holds it until the write finishes, so the `aw_ready` the FSM is waiting for never comes back.

Read FSM

State	Drives	Leaves When
`AXI_READ_IDLE`	`arvalid` while a read request is present	`ar_ready`
`AXI_WAIT_READ`	`rready`	`r_valid`

Three Timing Rules

All three are required. Each of them was a real bug during bring-up, and each failed silently.

The SRAM path stays combinational

Address, write data and byte strobes pass straight through with no register. The core-to-DSRAM path therefore adds nothing beyond the SRAM's own one-cycle read latency.

AXI address and data come straight from the LSU bus

The `valid` outputs assert in the same cycle the request arrives. If the address were held in a register, that register would only update at the *end* of that cycle — so whenever the crossbar accepted immediately, which its spill registers do when empty, the transaction would go out carrying the previous address (zero after reset).

Driving them combinationally is safe because `mem_stall` freezes the pipeline, which holds `mem_addr` and `mem_wdata` stable for the whole transaction.

`mem_stall` must be a wire, not a register

It has to go high in the same cycle the core makes the request. If it is registered it goes high one cycle later, and by then the core has already finished the instruction and moved on. The symptom is a program that runs impossibly fast, with loads returning old data.

Listing 3: Combinational handshake back to the core

```
wire axi_wr_done = (axi_write_state == AXI_WAIT_RESP) && m_axi_resp_i.b_valid;
wire axi_rd_done = (axi_read_state  == AXI_WAIT_READ) && m_axi_resp_i.r_valid;

wire axi_wr_busy = axi_req_active && mem_we && !axi_wr_done;
wire axi_rd_busy = axi_req_active && mem_re && !axi_rd_done;

mem_stall = sram_stall_needed | axi_wr_busy | axi_rd_busy;
mem_rdata = addr_in_sram_range ? sram_rdata
          : axi_rd_done        ? m_axi_resp_i.r.data   // forward on release cycle
                               : axi_rdata_reg;
```

Read data is forwarded combinationally on the cycle `r_valid` arrives, because that is the same cycle the stall drops and the core latches its result.

Error Handling

`mem_err` is raised on an address fault, or when a completed transaction returns a response other than OKAY. Like the stall, it must be combinational: a registered error would arrive after the core had already retired the access, so a bus fault would be missed entirely. The core turns `mem_err` into a load or store access fault at the MEM stage.

Latency

Access	Cost	Why
Instruction fetch from ISRAM	0 stall cycles	`imem_controller` pre-fetches, hiding the SRAM latency
Load from DSRAM	1 stall cycle	Minimum for a registered-output SRAM
Store to DSRAM	0 stall cycles	Nothing comes back; the stall is gated on `!mem_we`
MMIO read or write	≈ 3–4 cycles	AXI issue, crossbar address register, 2-cycle APB, response

Core to SoC Connections

`top.v` exposes three signal groups plus interrupts. There is no AXI master on the core itself and no request/grant handshaking on either memory port.

Instruction Fetch

Core Signal	Width	SoC Connection
`imem_sram_en`	1	`core_instr_req` into the demux
`imem_sram_addr`	30	Word address; SoC forms `{addr, 2'b00}`
`imem_sram_rdata`	32	Instruction word back from the demux
`imem_sram_we/wmask/wdata`	—	Unconnected; the fetch path is read-only

The core expects one-cycle read latency and provides no grant input. That is safe because it owns `sram_mux` port 0, which has unconditional priority.

Note

`imem_controller` assumes the instruction memory presents word 0 combinationally coming out of reset, with no startup bubble. Both `boot_rom` and `Instruction_Memory_new` satisfy this by resetting their address register to zero. Any replacement memory must do the same.

Data

`lsu_mem_req`, `lsu_mem_we`, `lsu_mem_addr`, `lsu_mem_wdata` and `lsu_mem_be` go out; `lsu_mem_rdata`, `lsu_mem_stall` and `lsu_mem_err` come back. This bus connects only to `hybrid_mem_interface`.

Control and Interrupts

Signal	Width	Connection
`clk`	1	`clk_i`
`rst`	1	`core_rst_n`, active low
`RESET_PC` (parameter)	32	`0x0000_0000`, the BootROM base
`fast_irq_i`	16	Peripheral interrupts
`timer_irq_i`	1	`timer0_irq_o[1]`
`ext_irq_i`, `sw_irq_i`	1 each	Tied low; there is no CLINT
`PCSrcE1`	1	Debug observation only

Constants That Must Agree

These values are declared in more than one place by necessity, since the modules are independently reusable. A mismatch is not caught at elaboration, so they must be changed together.

Constant	Declared In	Must Match
DSRAM base	`hybrid_mem_interface.DMEM_`	`xbar_addr_map[1]`, `bootloader.DSRAM_BASE_ADDR`
ISRAM base	`instr_fetch_demux.ISRAM_BASE`	`xbar_addr_map[0]`, `bootloader.ISRAM_BASE_ADDR`, `imem_sram_wrapper.IMEM_BASE_BYTE`
Reset vector	`top.RESET_PC`	`instr_fetch_demux.RESET_PC` and the BootROM image
MMIO window	`hybrid_mem_interface.MMIO_BASE/LIMIT`	`xbar_addr_map[2]`

CSR File

Machine mode only. Every address below is decoded; anything else reads as zero.

Address	Name	Access	Description
`0x300`	`mstatus`	R/W	WARL mask `0x0000188A`; `[3]` MIE, `[7]` MPIE, `[12:11]` MPP
`0x304`	`mie`	R/W	Interrupt enable
`0x305`	`mtvec`	R/W	Trap vector base
`0x340`	`mscratch`	R/W	Scratch register
`0x341`	`mepc`	R/W	Exception PC
`0x342`	`mcause`	R/W	Trap cause
`0x343`	`mtval`	R/W	Faulting address or instruction

Address	Name	Access	Description
0x344	mip	R	Interrupt pending; combines software bits with hardware inputs
0xB00	mcycle	R/W	Cycle counter, low word
0xB02	minstret	R/W	Retired-instruction counter, low word
0xB80	mcycleh	R/W	Cycle counter, high word
0xB82	minstreth	R/W	Retired-instruction counter, high word

Interrupt Sources

There is no CLINT. Peripheral interrupts arrive as Fast Local Interrupts in mip[31:16]; the standard timer, external and software bits are driven separately.

mip bit	mcause	Source	Core Port
19	19	UART interrupt	fast_irq_i[3]
18	18	SPI event	fast_irq_i[2]
17	17	I2C interrupt	fast_irq_i[1]
16	16	GPIO global interrupt	fast_irq_i[0]
11	11	Platform external	ext_irq_i, tied low
7	7	Timer	timer_irq_i from timer0_irq_o[1]
3	3	Software	sw_irq_i, tied low

Priority inside top.v is UART, then SPI, I2C, GPIO, external, timer, software. An interrupt is taken only when mstatus.MIE is set and some bit of mip & mie is high.

Instruction Set

The core targets RV32IM, Machine mode, with Zicsr. These opcodes are decoded by Main_Decoder.v and validated by opcode_check.v:

Opcode	Type	Instructions
0110111	LUI	LUI
0010111	AUIPC	AUIPC
1101111	JAL	JAL
1100111	JALR	JALR
1100011	BRANCH	BEQ, BNE, BLT, BGE, BLTU, BGEU
0000011	LOAD	LB, LH, LW, LBU, LHU
0100011	STORE	SB, SH, SW
0010011	OP-IMM	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
0110011	OP	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND and the M-extension MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU
1110011	SYSTEM	CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI, MRET
0001111	MISC-MEM	FENCE, accepted as legal and executed as a NOP

Verified in the Full SoC

Six instructions have been run in the complete system — fetched from ISRAM through the demux, decoded, executed, and their architectural effect checked:

Instruction	Encoding	What Was Verified
LUI	0x400010B7	Upper-immediate load, from both the BootROM and ISRAM
ADDI	0x00900113	Register-immediate add against x0
SW	0x0020A223	Store word, on both the MMIO path and the direct DSRAM path
LW	0x0040A183	Load word over MMIO with correct read data returned
BEQ	0x00310663	Taken and not-taken branch with the correct 2-cycle penalty
JAL	0x0080006F	Unconditional jump, including the BootROM to ISRAM transition

Important

The M-extension, sub-word accesses (LB, LH, LBU, LHU, SB, SH), JALR, AUIPC, the CSR instruction group and the trap/MRET path have not been exercised at system level. Given the unresolved width mismatches in rv32m_mul.v, the M-extension is the most important gap.